from the cache. We decided to keep the method private, for good reason, in our example, but let's now imagine that we need to have it as a public method, guaranteeing the option to clients to free space in the cache (maybe because of memory or garbage collection issues we need to shrink the cache dynamically – we certainly would rather not restart it altogether and lose all of its content).

In that case, we need to make `evictOneEntry` synchronized as well, and hence acquire a write lock inside of it. With a non-reentrant lock, `evictOneEntry` would try to acquire the write lock, but `set` has already acquired it, and can't free it till `evictOneEntry` finishes. Long story short, our thread is stuck waiting for a resource that will never be freed, and all other threads will soon be stuck as well. That is basically a recipe for deadlock, and it's that easy to get yourself in such a situation: that's why you should be careful with synchronization mechanisms.

With reentrant locks instead, since both `evictOneEntry` and `set` belong to the same thread, `evictOneEntry` on line #1 is able to acquire the lock that is held by set, and can go on with its execution: deadlock avoided, at least this time!

7.7.5 Read Locks

We haven't yet talked about read locks: we mentioned how important they are for performance, but why? Let's take a look at our revisitation of the `get` method.

We acquire a read lock before reading data from cache's internal fields, and release it as the very last step before